A truncated distance bound for cyclic lifted-product quantum codes
Small-support geometry and checked integer certificates

25 October 2026

Abstract

We prove a sufficient condition for the ordinary Pauli distance of a symmetric cyclic lifted-product code to satisfy $d_Q \geq \min(d_C, 12)$. The base is a normalized, fully populated 3×4 monomial matrix over $\mathbb{F}_2[x]/(x^L - 1)$. The conditions are classical Tanner girth at least eight, absence of a full column partition, exclusion of three equal values of each three-row pairing form, and transpose-code distance at least d_C. The proof works for arbitrary cyclic lift length, including composite lengths. A source-kernel argument reduces every mixed support of weight at most eleven to three block-count profiles, of weights seven, ten and eleven. Actual support encodings and 183,312 checked integer certificates eliminate the non-stabilizer possibilities. Six exceptional certificates instead force a classical word of weight at most ten, which suffices for the distance bound. Lean checks the complete implication for the actual expanded binary matrices, both full stabilizer spans, both CSS sectors and Pauli union weight, together with the literal distance minima and the transpose infinity convention. The result is a bounded sufficient condition motivated by Raveendran, Declercq and Vasić's question about $d_Q = d_C$; it does not settle that broader question.

1 The theorem and the source question

Let $L \geq 2$ and $G = \mathbb{Z}/L\mathbb{Z}$. A matrix $E : \{0,1,2\} \times \{0,1,2,3\} \to G$ specifies the monomial base $B_{ic} = x^{E_{ic}}$. Every entry is present: exponent zero means an identity circulant. We normalize

$$E_{0c} = E_{i0} = 0. \tag{1}$$

The associated symmetric lifted product has $25L$ physical qubits and binary checks obtained from

$$C = \mathrm{expand}(B), \qquad H_{\mathsf{X}} = \mathrm{expand}[B \otimes I_4 \mid I_3 \otimes B^*], \qquad H_{\mathsf{Z}} = \mathrm{expand}[I_4 \otimes B \mid B^* \otimes I_3]. \tag{2}$$

Here star transposes and sends x to x^{-1}. Section 2 fixes the coefficient convention and the full logical quotient.

We use two explicit entry tests. The *column test* excludes any partition of all four distinct columns a, b, c, d for which

$$E_{ia} + E_{ib} = E_{ic} + E_{id} \quad \text{for every row } i. \tag{3}$$

For distinct rows i, j, k, put

$$F_{ijk}(c) = E_{ic} + E_{jc} - 2E_{kc}. \tag{4}$$

The *triple-row test* says that no three distinct columns have the same value of F_{ijk}. In particular, it excludes a full-row RCPC, since four zero values would contain three equal values. These tests are stated directly in the exponent group; their exact relation to the source conventions is recorded in Appendix B.

Theorem 1.1 (Truncated source distance)**.** *For the construction* (2)*, assume normalization* (1)*, classical Tanner girth at least eight, the column and triple-row tests, and* $d_{C^\mathsf{T}} \geq d_C$*. Then*

$$d_Q \geq \min(d_C, 12). \tag{5}$$

The transpose distance is $+\infty$ *when* $\ker C^\mathsf{T} = 0$*. Quantum distance is measured with both entire stabilizer row spans and ordinary Pauli union weight. The implication holds for arbitrary cyclic modulus; neither oddness nor primality is assumed.*

The complete distance statement is `TruncatedDistance.source_distance_bound_twelve`. Its general predicate form, `TruncatedDistance.source_pauli_bound_twelve`, assumes that every nonzero classical and transpose kernel word has weight at least D, and concludes the Pauli bound $\min(D, 12)$. The proof of Theorem 1.1 is completed in Section 6. Appendix A maps every argument below to the checked declarations; the following Lean appendices print complete central proof bodies and link every imported research module.

1.1 What is answered

Raveendran, Declercq and Vasić [1, Eq. (9), Definitions 2–5 and Section VI] ask for sufficient base constraints ensuring equality of the quantum and underlying classical distances. Their partition constraints motivate examining logical supports near the stabilizer-generator scale. For a 3×4 base a generator has weight seven, whereas a mixed logical can involve more coordinates. A previously established counterfamily [6] has classical girth eight, no full RCPC and $d_Q \leq 11 < 12 \leq d_C$, even when its transpose distance is at least d_C. This motivates controlling partial row pairing as well as full partitions.

The triple-row condition and the truncated target are the bounded criterion studied here, rather than a numbered conjecture attributed to the source paper. The distinction matters: a lower bound cut off at twelve leaves all higher-weight logical supports untreated and supplies no upper bound on d_Q.

Question	Source and locator	Answer
Which base constraints ensure $d_Q = d_C$?	Raveendran–Declercq–Vasić, arXiv:2503.07567v1, 10 March 2025, Section VI.	Remains open in general here.
Does the strengthened normalized 3×4 criterion imply $d_Q \geq \min(d_C, 12)$?	Project-proposed bounded follow-up to that question; construction Eq. (9), entry conventions Definitions 2–5.	Yes: Theorem 1.1, including actual source matrices and complete stabilizer quotients.
Can the finite algebraic obstructions be interpreted for composite lifts?	The proof obligation for this criterion.	Yes: integer identities are interpreted without division, and the six torsion-five alternatives are classical low-weight words (Lemma 5.2).

1.2 Comparison and reference chronology

Pairing selected coordinates at checks, passing to a colored triangle-free normal graph and using exact integer consistency certificates are related prior methods in Okada and Kasai [3, Sections 3.4 and 3.7]. Their inspected v2 proves distance eighteen for one $[[34542, 23032, 18]]$ code and an affine-lift ceiling for a different fixed finite-field base at prime lift degree greater than nineteen. Those statements do not establish the normalized 3×4 cyclic criterion here. The contribution of the present proof is its source-complete reduction and certificate interpretation for this family at every cyclic modulus, with the full logical quotient.

Lin and Pryadko [2, Section IV.6, Statement 12] give distance bounds for two-block group-algebra codes involving a support-intersection subgroup. Their two-element construction and bound do not imply this matrix-base criterion. Lee [4] supplies exact code-specific distance searches and equivalence reductions for small two-block codes; those computations are useful background, rather than premises of the universal implication here. The tree counting behind paired checks and Mantel's triangle-free edge bound are classical ingredients; related stopping-set tree bounds are discussed by Kelley et al. [5]. The actual matrix and quotient translations are included in the checked proof.

The central chronology is shown below; Appendix B.1 specifies coverage and date precision. Calendar ages are not durations of an unsolved problem or measurements of proof effort.

Source	Earliest date	Inspected version	Age to 25 Oct. 2026
Raveendran et al. [1]	10 Mar. 2025	v1, 10 Mar. 2025	594 days
Lin–Pryadko [2]	28 June 2023	v1, same date	1,215 days
Okada–Kasai [3]	25 June 2026	v2, 5 Sept. 2026	122 days; 50 from v2
Lee [4]	9 Oct. 2026	v2, 12 Oct. 2026	16 days; 13 from v2
Kelley et al. [5]	2004 (year only)	ISIT 2004	About 22 years

2 Source matrices, kernel words and the full quotient

A classical variable is $(c, s) \in \{0, 1, 2, 3\} \times G$ and a classical check is $(i, t) \in \{0, 1, 2\} \times G$, with coefficient one precisely when $t = s + E_{ic}$. Left product coordinates are $U(j, k, s)$, with $j, k \in \{0, 1, 2, 3\}$; right coordinates are $V(\ell, i, s)$, with $\ell, i \in \{0, 1, 2\}$. The actual Z-check equation at (j, i, t) is

$$\sum_k U(j, k, t - E_{ik}) + \sum_\ell V(\ell, i, t + E_{\ell j}) = 0 \quad \text{in } \mathbb{F}_2. \tag{6}$$

A left coordinate meets three checks and a right coordinate meets four. Both check matrices have $12L$ rows and $25L$ columns.

For binary vectors x, z on these physical coordinates, let

$$H_{\mathsf{Z}} x = 0, \qquad H_{\mathsf{X}} z = 0, \qquad (x, z) \notin \operatorname{row}(H_{\mathsf{X}}) \times \operatorname{row}(H_{\mathsf{Z}}). \tag{7}$$

Their Pauli weight is $|\operatorname{supp}(x) \cup \operatorname{supp}(z)|$. Every linear combination of every stabilizer row is allowed in (7); no restricted collection of generators is substituted. The quantum distance is the minimum of these weights. Equivalently, one may take the extended-natural infimum and assign $+\infty$ if there is no nontrivial pair. The lower bound applies to that convention as well as the usual positive-logical-dimension regime.

The classical distance is the least nonzero weight in $\ker C$, and the transpose distance uses $\ker C^{\mathsf{T}}$. A pure-left source kernel word splits into classical kernel words, one for each j; a pure-right word splits into transpose kernel words, one for each i. Thus a nonzero pure word cannot have weight below D if both classical lower predicates hold. The checked pure-sector statements are used in Lemma 6.1.

3 Small supports have only three mixed profiles

Lemma 3.1 (Actual incidence and paired checks)**.** *Classical no-four/no-six implies no-four/no-six for the actual Z-check incidence of (2). In a source kernel word of weight at most eleven, every check meets its support in zero or two coordinates.*

Proof. The coefficient incidence is exactly the relation in (6). Two coordinates of the same product block sharing two checks give a classical four-cycle; a mixed pair has only one check type in common. A six-cycle contained in one block gives a classical six-cycle. In a mixed six-cycle, the lone coordinate

from one block would have to use the same check type twice, contradicting distinctness of the cycle checks.

Kernel parity makes each selected check degree even. If a check met at least four selected coordinates, choose four. Each has at least two other checks; each of those needs another selected neighbor. Four-cycle exclusion makes the eight other checks distinct, and four-/six-cycle exclusion makes their eight required support neighbors distinct from one another and from the four original coordinates. The support would have at least twelve elements. The actual source-degree and short-cycle premises of this tree argument are proved in `ActualIncidence`; its conclusion is `TruncatedDistance.source_kernel_zero_or_two`. □

At each occupied check, join its two selected coordinates. This normal graph is simple by four-cycle exclusion and triangle-free by six-cycle exclusion. Its left and right vertex degrees are three and four. Set

$$a_j = |\operatorname{supp} U \cap \{\text{first index } j\}|, \qquad b_i = |\operatorname{supp} V \cap \{\text{second index } i\}|. \tag{8}$$

Lemma 3.2 (Complete mixed count classification)**.** *For a source kernel word of weight at most eleven with both blocks nonempty, the separately sorted counts are exactly one of*

weight	a	b
7	$(1,1,1,1)$	$(1,1,1)$
10	$(2,2,2,0)$	$(2,2,0)$
11	$(5,1,1,1)$	$(1,1,1)$.

(9)

Proof. Sum (6) over the fiber variable. For every j, i it gives $a_j + b_i = 0 \pmod 2$, so all seven counts have one common parity. Let m_{ji} count internal left-left normal edges in bucket (j, i) and c_{ji} count cross edges there. Socket counting gives $a_j = 2m_{ji} + c_{ji}$ and $c_{ji} \le b_i$. Distinct internal edges of a fixed left group form a triangle-free graph on a_j vertices. Mantel's bound gives

$$\sum_i \frac{\max(a_j - b_i, 0)}{2} \le \left\lfloor \frac{a_j^2}{4} \right\rfloor, \qquad \sum_j \frac{\max(b_i - a_j, 0)}{2} \le \left\lfloor \frac{b_i^2}{4} \right\rfloor. \tag{10}$$

All differences being divided by two are even. The analogous right-group argument proves the second inequality. There are forty-four sorted common-parity profiles with both blocks nonempty and total at most eleven. An exhaustive bounded-list enumeration proves completeness, and checking (10) on those profiles leaves exactly (9). The enumeration and inequalities are both checked from the actual source kernel in `TruncatedDistance.source_mixed_count_profiles`; the independent socket classification agrees. The enumeration is over integer counts, not sampled bases or lift lengths. □

4 Canonical source supports and finite encodings

Lemma 4.1 (Transport preserves the complete problem)**.** *Row/column permutation followed by re-normalization preserves the entry tests, classical and transpose lower predicates, source kernel, full stabilizer membership, weight and group counts. Hence the missing weight-ten groups may be placed at zero and the exceptional weight-eleven group at index three.*

Proof. Permutations relabel both tensor factors. For a gauge change $E'_{ij} = E_{ij} + r_i + c_j$, use the fiber bijections

$$U'(j,k,s) = U(j,k,s + c_j + c_k), \qquad V'(\ell,i,s) = V(\ell,i,s - r_\ell - r_i). \tag{11}$$

Substitution checks the kernel equation; stabilizer coefficients transform by $w'(i,j,s) = w(i,j,s - r_i + c_j)$. These are coordinate bijections, so both full spans and weight are preserved. Classical and transpose kernels have the corresponding coordinate transports.

Re-normalizing a permuted base subtracts one column-independent constant from each row form, preserving equality of three values. For the column partition form $A_i = E_{ia} + E_{ib} - E_{ic} - E_{id}$ it replaces A_i by $A_i - A_r$. The original zero row gives $A_0 = 0$, so simultaneous vanishing is preserved. Girth is preserved by the source incidence bijections. Finally the count classification supplies the unique missing or exceptional group and the values of every other group. The complete transports are in `ProfileTransport`; the marker choices are in `CountCanonical`. Thus canonical placement is derived, rather than an additional hypothesis. □

Lemma 4.2 (Singleton labels and the constant-row branch)**.** *The singleton groups in the weight-seven and weight-eleven profiles have label maps that are either both constant or both injective. The constant case is in the full stabilizer span when the word is below the classical lower bound. The weight-seven profile is always a stabilizer under the entry tests.*

Proof. Write a singleton left coordinate as (j, k_j, s_j) and a right coordinate as (ℓ_i, i, t_i). Their actual cross checks give

$$s_j + E_{i,k_j} = t_i - E_{\ell_i,j}. \tag{12}$$

If two k_j agree, subtraction forces a column difference to be independent of i; classical four-cycle exclusion forces all ℓ_i to agree. The converse is symmetric. Hence the maps are both constant or both injective.

For constants K, H, equation (12) supplies a common z with $s_j = z - E_{Hj}$ and $t_i = z + E_{iK}$. These are precisely the coordinates of the complete X-stabilizer row indexed by (H, K, z). At weight seven this is the whole word. At weight eleven, adding that row cancels the three singleton left coordinates and all three right coordinates; the remainder is pure-left of weight at most six. Below the classical lower bound it vanishes, proving full stabilizer membership. The checked source statement includes the right coordinates and all stabilizer coefficients.

In the injective weight-seven case, the label maps are permutations of four columns and three rows. The resulting $24 \cdot 6 = 144$ patterns are all certified forbidden by the entry/girth tests through the integer argument of Section 5. Actual source realizations are constructed in `SourceSeven`; none is postulated as a new premise. □

Lemma 4.3 (Complete weight-ten encoding)**.** *Every actual weight-ten kernel support with counts in* (9)*, after the transport of Lemma 4.1, realizes an index in a catalog of size* $124,416$*.*

Proof. At the missing right row zero, each remaining two-coordinate left group must be internally paired. Its two fibers agree by normalization, and its two column labels differ. The same two coordinates cannot be internally paired at another row, since that would give a classical four-cycle. Similarly each right pair is internally matched at missing left column zero, with equal fibers and different row labels. Every other bucket is a bijection between two-element sets and has exactly two matching choices.

The three increasingly ordered column pairs are selected from six possibilities each, the two increasingly ordered row pairs from three possibilities each, and the six cross-bucket matchings from two possibilities each. Thus the finite decoder has $6^3 3^2 2^6 = 124,416$ indices. The geometry is extracted from actual supports; rank bounds, decoder identities and every source incidence are proved before applying the catalog. The complete interface is `TruncatedDistance.actual_canonical_ten_support_encodes`. □

Lemma 4.4 (Complete weight-eleven encoding)**.** *For a weight-eleven support in* (9)*, the constant-label branch is covered by Lemma 4.2. Every injective-label support realizes an index in a catalog of size* $58,752$*.*

Proof. The exceptional left group has five vertices and six internal normal edges: its fifteen sockets have exactly three cross partners. A triangle-free five-vertex graph with six edges is $K_{2,3}$. Its two centers

use all three row colors internally; each leaf has one missing color and one cross partner. The missing colors are distinct because the right singleton in any row has only one socket at the exceptional first-index group. Index the leaves by those missing colors. There are two compatible row-color orientations. Adjacent internal vertices cannot have the same second column label, since their common source check would then force identical coordinates.

The complete finite label table has 204 admissible center/leaf tuples. There are 24 injective singleton-column maps, six row permutations and two orientations, giving $24 \cdot 6 \cdot 204 \cdot 2 = 58,752$ decoder indices. The actual $K_{2,3}$ numbering, colors, missing-row permutation, singleton partners and eighteen source incidences are extracted in the source proof. `TruncatedDistance.source_canonical_eleven_stabilizer` assembles this encoding with the complete constant-label branch and the certified exclusions. □

5 Integer certificates for every encoded pattern

Lemma 5.1 (Universal interpretation of the certificates)**.** *A checked integer row identity for a catalog pattern implies its specified forbidden equation in G, whenever that pattern has an actual source realization. This implication requires no division and holds at every cyclic modulus.*

Proof. Let y consist of the six normalized exponents and the w vertex fibers. Every matching edge gives a homogeneous integer equation in these variables, with signs fixed by (6). The full matching matrix M uses eighteen row slots, padding unused slots when necessary. A certificate for a target row f is an integer vector a satisfying $f_j = \sum_r a_r M_{rj}$ for every variable j. In every additive group,

$$\sum_j f_j y_j = \sum_r a_r \Big(\sum_j M_{rj} y_j \Big) = 0.$$

All fiber columns are included in the checked identity. Certificates involving several targets provide each identity separately.

The checker verifies these integer equalities, and `KernelCatalog.checked_fast_equations` interprets its Boolean success through `TruncatedDistance.integer_certificate_sound`. An actual realization supplies the matching equations. The sixty-four target outcome types are interpreted as short cycles, a prohibited entry pattern, small cyclic torsion, or the classical alternative in Lemma 5.2. This final source exclusion is `TruncatedDistance.certified_pattern_has_no_source_realization`. □

Lemma 5.2 (Cyclic torsion and the six classical alternatives)**.** *The small row/column torsion outcomes contradict classical no-four. Each of the six remaining weight-ten alternatives instead forces a nonzero classical word of weight at most ten, contradicting $10 < d_C$.*

Proof. For distinct rows, the four differences across columns are distinct under no-four; for distinct columns, the three row differences are distinct. For positive n, the n-torsion subgroup of G has at most n elements. Consequently four distinct row differences cannot all be killed by $1 \le n \le 3$, and three distinct column differences cannot all be killed by two.

Each exceptional weight-ten pattern forces $5E_{1c} = 5E_{2c} = 0$ for some $c \neq 0$. Set $K = \{s \in G : 5s = 0\}$ and take the binary indicator of $\{0, c\} \times K$. It is nonzero and has weight $2|K| \le 10$. In each classical check its two contributions cancel: $E_{i0} = 0$, and translation by $E_{ic} \in K$ preserves K. This is an actual C-kernel word, proved by `TruncatedDistance.source_classical_word_le10`. The alternative contradicts a proposed weight-ten support strictly below d_C; it is not a qualifying counterexample. In particular the argument never divides by five, including when $5 \mid L$. □

Proposition 5.3 (Complete quantified catalog coverage)**.** *Every decoder index at weights seven, ten and eleven has a kernel-checked certificate. The catalog sizes are respectively* 144, 124,416 *and* 58,752*.*

Proof. Each closed Lean case proves that the full integer-identity checker returns true for its explicit certificate. A balanced interval proof assembles the closed cases into $\forall p : \mathrm{Fin}(N), \exists c, \mathrm{check}(p, c)$. The checked append lemma composes adjacent intervals and covers both branches of the index split. The resulting uniform statements are `TruncatedDistance.seven_certificates_complete`, `KernelCatalog.complete_coverage10` and `KernelCatalog.complete_coverage11`. An independent standard-library verifier reconstructs every matching matrix and integer identity; it agrees on all 183,312 indices, but is not a premise of these Lean theorems. □

Certificate outcome	Weight 7	Weight 10	Weight 11
Classical four-cycle	109	124,129	57,282
Classical six-cycle	16	262	1,008
Forbidden three-column row form	9	9	342
Full column partition	3	3	66
Small cyclic row torsion	1	1	0
Column two-torsion	6	6	54
Classical word of weight at most ten	0	6	0
Total	144	124,416	58,752

These are certificate classifications of the finite decoder indices. They are not counts of inequivalent codes, lift lengths or logical operators. Discovery used integer algebra; checking uses the full source matching rows, including fiber variables. No finite case remains unclassified.

6 The full source bound and ordinary Pauli distance

Lemma 6.1 (Global full-span reduction)**.** *Assume the source entry/girth tests and classical/transpose lower predicates at D. Every source Z-kernel word of weight below* $\min(D, 12)$ *lies in the entire row span of H_X.*

Proof. A pure-block word below D is zero by the actual classical or transpose kernel bound. Otherwise Lemma 3.2 leaves only weights seven, ten and eleven. Lemma 4.1 supplies canonical placement and preserves every hypothesis and the full quotient. The weight-seven case is a stabilizer by Lemma 4.2. For weight ten, Lemma 4.3 gives an actual realization of a certified pattern. Its outcome contradicts a source test, or forces $d_C \leq 10$ while the word has weight ten below D. For weight eleven the constant case is a stabilizer; the injective case is excluded by Lemma 4.4 and the complete catalog. Transport the resulting full-span statement back to the original source coordinates. The case split is printed in the Lean appendix as `TruncatedDistance.source_kernel_below_twelve_of_coverage`; `TruncatedDistance.source_kernel_below_twelve_is_stabilizer` discharges its finite-coverage premise with Proposition 5.3. □

Lemma 6.2 (Both sectors and union weight)**.** *An X-logical lower bound b for this source construction gives the same Z-logical lower bound and the ordinary Pauli lower bound b.*

Proof. Swap the two finite indices within each tensor-coordinate block. This involutive coordinate permutation exchanges the actual X/Z kernel equations and full spans and preserves support size. A nontrivial Pauli pair has a component outside its corresponding full span. That component is a pure logical, and its weight is at most the union weight. Apply the sector bound and support inclusion. The complete union-weight proof is `TruncatedDistance.pauli_bound_of_x_bound`. □

Proposition 6.3 (Literal distance minima)**.** *The classical minimum is attained and satisfies its universal lower predicate. The transpose minimum is infinity for a zero transpose kernel, and a comparison $D \leq d_{C^\intercal}$ implies the corresponding lower predicate. For the full Pauli quotient, $b \leq d_Q$ is equivalent to saying that every nontrivial pair has union weight at least b.*

Proof. The actual $3L \times 4L$ classical matrix has a nonzero kernel word, as proved in `GeneralLPPhysical`; define its distance by the least attained natural weight. Leastness gives the universal predicate. Define transpose and Pauli distances as extended-natural infima of their actual nonzero-word or nontrivial-pair weights. The infimum of the empty set is infinity. An infimum is at most each member, and a lower bound on every member is at most the infimum. These properties prove the required implications and the Pauli equivalence. All three definitions and the bridges are checked in `DistanceMinima`; the public theorem retains the complete quotient. □

Proof of Theorem 1.1. Set $D = d_C$. Proposition 6.3 supplies the classical lower predicate and obtains the transpose predicate from the assumed comparison. Lemma 6.1 excludes every X logical below $\min(D, 12)$; Lemma 6.2 gives the ordinary Pauli bound. Apply the minimum characterization in Proposition 6.3. This is exactly the proof body of `TruncatedDistance.source_distance_bound_twelve`. □

7 Implications and remaining question

The theorem controls the entire small-support range relevant to the truncation: every hypothetical nontrivial word below $\min(d_C, 12)$ is excluded in the actual source model. The integer argument remains valid at even and composite lift lengths; the torsion-five alternatives explain why a certificate need not be a pure entry obstruction to finish a distance lower bound.

The conclusion is sufficient and one-sided. It does not characterize all bases with preserved distance, give an exact d_Q, or settle the source's unrestricted $d_Q = d_C$ design question. No decoder-performance conclusion follows from it. Higher-weight supports need further arguments. A concrete possible follow-up is the already-studied $L = 13$ base with classical distance fourteen and a saved quantum upper witness of weight fourteen. Its computational exclusion through thirteen is separate evidence; a faithful Lean distance-fourteen certificate would still need the concrete hypotheses, upper witness and weight-twelve/thirteen exclusion. That finite follow-up is not required for Theorem 1.1.

A Proof coverage and reproducibility

The checked toolchain is Lean 4.36.0-rc1, commit `c9afcddf514a6772bd4d649f7cfc04eff0b9f15e`, with Mathlib commit `5f7b4198d22a9c82e1c698a379334fe128095c82`. The immutable proof snapshot is `3ae21c0719cedbd928f4f7882f9b9e266567605e`. The main source-distance theorem and source-scope audit have 269 research imports, all admitted against matching source and log hashes. Their reported axioms are contained in `propext`, `Classical.choice` and `Quot.sound`. No native decision oracle or unproved catalog premise is admitted. The dependency cache is pinned; Mathlib itself is not presented as newly rebuilt here.

The table maps every theorem/proof in the mathematical exposition to its faithful formal counterpart, including the support constructions, finite coverage and literal minima. `ReportAudit.lean` checks the named declarations and prints their axiom closures. The following appendices print complete central declarations extracted from the immutable checked sources. The large finite case files are linked in full in Appendix I.

Manuscript proof	Checked Lean declarations
1.1: Literal source-distance bound	`TruncatedDistance.source_distance_bound_twelve`
3.1: Actual coefficients, short cycles and paired checks	`TruncatedDistance.classical_incidence`; `TruncatedDistance.quantum_incidence`; `TruncatedDistance.quantum_no_four`; `TruncatedDistance.quantum_no_six`; `TruncatedDistance.source_kernel_zero_or_two`
3.2: Parity, local Mantel bounds and exhaustive profiles	`TruncatedDistance.common_block_parity`; `TruncatedDistance.common_count_parity`; `TruncatedDistance.source_left_local_mantel`; `TruncatedDistance.source_right_local_mantel`; `TruncatedDistance.count_cases_length`; `TruncatedDistance.classified_of_mem_countCases`; `TruncatedDistance.source_mixed_count_profiles`
4.1: Full source transport and canonical placement	`TruncatedDistance.normPerm_criterion`; `TruncatedDistance.normPerm_kernel`; `TruncatedDistance.normPerm_span`; `TruncatedDistance.normPerm_weight`; `TruncatedDistance.ten_left_marker`; `TruncatedDistance.ten_right_marker`; `TruncatedDistance.eleven_left_marker`
4.2: Constant or injective singleton labels and full-row cancellation	`TruncatedDistance.actual_singleton_label_alternative`; `TruncatedDistance.constant_singleton_rows`; `TruncatedDistance.single_exception_cancellation`; `TruncatedDistance.source_seven_singleton_profile_stabilizer`
4.3: Actual paired weight-ten source encoding	`TruncatedDistance.source_ten_geometry`; `TruncatedDistance.actual_canonical_ten_support_encodes`; `TruncatedDistance.tenRank_bound`
4.4: Actual colored K2,3 and weight-eleven source encoding	`TruncatedDistance.exceptional_group_is_k23`; `TruncatedDistance.eleven_geometry_encodes`; `TruncatedDistance.source_canonical_eleven_stabilizer`
5.1: Full integer identities, universal interpretation and source obstruction	`TruncatedDistance.integer_certificate_sound`; `KernelCatalog.checked_fast_equations`; `TruncatedDistance.certified_pattern_has_no_source_realization`; `TruncatedDistance.source_target_obstruction`
5.2: Classical alternatives without division	`TruncatedDistance.torsion_card_le`; `TruncatedDistance.torsionWord_source_kernel`; `TruncatedDistance.source_classical_word_le10`; `TruncatedDistance.row_small_torsion_impossible`; `TruncatedDistance.column_two_torsion_impossible`
5.3: Quantified certificate coverage	`TruncatedDistance.seven_certificates_complete`; `KernelCatalog.complete_coverage10`; `KernelCatalog.complete_coverage11`; `KernelCatalog.covered_append`

Manuscript proof	Checked Lean declarations
6.1: Pure sectors and complete full-span reduction	`TruncatedDistance.pure_left_zero_below_classical`; `TruncatedDistance.pure_right_zero_below_transpose`; `TruncatedDistance.source_kernel_below_twelve_of_coverage`; `TruncatedDistance.source_kernel_below_twelve_is_stabilizer`
6.2: Actual sector symmetry and ordinary union weight	`TruncatedDistance.full_span_swap`; `TruncatedDistance.kernel_swap`; `TruncatedDistance.swap_weight`; `TruncatedDistance.pauli_bound_of_x_bound`; `TruncatedDistance.source_pauli_bound_twelve`
6.3: Actual classical/transpose/quantum minima	`TruncatedDistance.source_classical_minimum_realized`; `TruncatedDistance.source_classical_minimum_lower`; `TruncatedDistance.source_transpose_zero_kernel_infinity`; `TruncatedDistance.source_quantum_minimum_lower_iff`

The proof dependencies can be read in the following order:

Actual source matrices and incidence
↓
Paired checks, parity and local triangle-free edge bounds
↓
Three count profiles; actual weight-seven/ten/eleven encodings
↓
Full integer identities and quantified finite coverage
↓
Permutation/normalization transport and full-span reduction
↓
Both sectors, union weight and literal distance minima

From the investigation directory, run the independent verifier and rebuild with pinned dependencies in a fresh object directory:

```
python3 verify_catalog.py
python3 replay_round.py --mathlib /path/to/pinned/mathlib4 \
  --lean-bin /path/to/pinned/lean/bin --build-dir /path/to/fresh/objects \
  DistanceTwelve CriterionScope
python3 verify_round.py --round-number 2 --final
```

The archive preserves relative source relationships and all required research imports. Generators, encoded certificates, interval plans and build records are included. Discovery outputs and prior unsuccessful classifications remain in the research repository; they are not proof premises. The standalone report build and integrity checker are described in the bundled `README.txt`. Investigation effort and report preparation timestamps are recorded outside the mathematical exposition in the provenance records.

B Source scope and reference ledger

The source construction is Raveendran–Declercq–Vasić v1, equation (9). The column condition is its full-column RCPC exclusion at four columns (Definition 2). At three rows, full-row RCPC has the row form (4) equal to zero at every column (Definition 5). The stronger triple-row test excludes that case; `TruncatedDistance.no_full_row_of_no_triple` checks the implication. Thus omission of a separate full-row field in the final Lean statement removes only a redundant hypothesis. The formal theorem uses a positive cyclic modulus; the displayed $L \geq 2$ scope is contained in it. The actual coefficient matrices, not just a support surrogate, determine all kernel and span predicates.

B.1 Chronology and bounded prior-art coverage

The manuscript date is 25 October 2026. Exact calendar deltas in the table near the introduction are differences to that date, using the earliest arXiv posting and the inspected version date separately. The source histories are linked in the bibliography and recorded in `reference-chronology.json`. The 2004 stopping-set reference has only year precision here, so its age is approximate. A source's age is not a claim that its broad question was unsolved throughout that interval.

The source comparison reuses the completed full-text screens in the source ledger. It includes the original question, full-theorem comparisons and harvested questions from encountered papers. The report metadata pass rechecks dates of those same versions; it creates no new paper/question cohort. Public GitHub and primary-source searches were bounded. Native X access was unavailable; indexed discovery was not treated as authenticated coverage. Uninspected leads remain named gaps in the ledger. These limits concern source coverage, rather than a premise of the mathematical theorem.

References

[1] N. Raveendran, D. Declercq and B. Vasić, *On the Minimum Distances of Finite-Length Lifted Product Quantum LDPC Codes*, arXiv:2503.07567v1, 10 March 2025. Construction Eq. (9), Definitions 2–5, Sections V-B and VI.

[2] H.-K. Lin and L. P. Pryadko, *Quantum two-block group algebra codes*, arXiv:2306.16400v1, 28 June 2023. Section IV.6, Statement 12.

[3] K. Okada and K. Kasai, *Rate-2/3 Girth-8 (3,18)-Regular Quantum LDPC Codes from Two-Branch Finite-Field Bases and CPM Lifts*, arXiv:2606.27130v2, first posted 25 June 2026; inspected v2, 5 September 2026.

[4] G. Lee, *Bicycle Race: small two-block codes on the Pareto frontier*, arXiv:2610.13880v2, first posted 9 October 2026; inspected v2, 12 October 2026.

[5] C. Kelley, D. Sridhara, J. Xu and J. Rosenthal, *Pseudocodeword Weights and Stopping Sets*, ISIT 2004. Author-hosted paper; Theorem III.1 and its following note.

[6] *Distance loss in girth-eight cyclic lifted-product codes: An infinite counterfamily with checked Lean proofs*, repository report, 25 October 2026. Report and source links.

C Source matrices, full spans and exact distances

The source interface fixes actual binary coefficients, every stabilizer row combination and the extended-natural minimum convention.

`GeneralLP`: HX

Complete checked declaration at the immutable proof commit.

```
def HX (e : Fin 3 → Fin 4 → ZMod L) (c : XCheck L) (q : Qubit L) : ZMod 2 := match q with
  | .inl ((i,j),s) => if c.1.2=j then P (e c.1.1 i) c.2 s else 0
  | .inr ((i,j),s) => if c.1.1=i then P (-e j c.1.2) c.2 s else 0
```

`GeneralLP`: HZ

Complete checked declaration at the immutable proof commit.

```
def HZ (e : Fin 3 → Fin 4 → ZMod L) (c : ZCheck L) (q : Qubit L) : ZMod 2 := match q with
  | .inl ((i,j),s) => if c.1.1=i then P (e c.1.2 j) c.2 s else 0
  | .inr ((i,j),s) => if c.1.2=j then P (-e i c.1.1) c.2 s else 0
```

`GeneralLP`: InSpanX

Complete checked declaration at the immutable proof commit.

```
def InSpanX (e : Fin 3 → Fin 4 → ZMod L)
    (u : Fin 4 → Fin 4 → ZMod L → ZMod 2) (v : Fin 3 → Fin 3 → ZMod L → ZMod 2) : Prop :=
  ∃ w : XCheck L → ZMod 2, ∀ q, join u v q=∑ c, w c*HX e c q
```

`GeneralSymmetry`: fullSpanX

Complete checked declaration at the immutable proof commit.

```
def fullSpanX (e : Fin 3 → Fin 4 → ZMod L) (x : Qubit L → ZMod 2) : Prop :=
  ∃ w : SourceModel.XCheck L → ZMod 2, ∀ q, x q=∑ c, w c*GeneralLP.HX e c q
```

`GeneralSymmetry`: fullSpanZ

Complete checked declaration at the immutable proof commit.

```
def fullSpanZ (e : Fin 3 → Fin 4 → ZMod L) (x : Qubit L → ZMod 2) : Prop :=
  ∃ w : SourceModel.ZCheck L → ZMod 2, ∀ q, x q=∑ c, w c*GeneralLP.HZ e c q
```

`DistanceMinima`: sourceClassicalHasWeight

Complete checked declaration at the immutable proof commit.

```
def sourceClassicalHasWeight (e : Fin 3 → Fin 4 → ZMod L) (n : ℕ) : Prop :=
  ∃ x : Fin 4 × ZMod L → ZMod 2,
    (∀ a,(∑ q,GeneralLP.C e a q*x q)=0) ∧ x≠0 ∧ (CoverWeight.support x).card=n
```

`DistanceMinima`: sourceClassicalDistance

Complete checked declaration at the immutable proof commit.

```
noncomputable def sourceClassicalDistance (e : Fin 3 → Fin 4 → ZMod L) : ℕ :=
  Nat.find (source_classical_weight_exists e)
```

`DistanceMinima`: sourceTransposeHasWeight

Complete checked declaration at the immutable proof commit.

```
def sourceTransposeHasWeight (e : Fin 3 → Fin 4 → ZMod L) (n : ℕ∞) : Prop :=
  ∃ y : Fin 3 × ZMod L → ZMod 2,
    (∀ q,(∑ a,GeneralLP.C e a q*y a)=0) ∧ y≠0 ∧ n=((CoverWeight.support y).card : ℕ∞)
```

`DistanceMinima`: sourceTransposeDistance

Complete checked declaration at the immutable proof commit.

```
noncomputable def sourceTransposeDistance (e : Fin 3 → Fin 4 → ZMod L) : ℕ∞ :=
  sInf {n | sourceTransposeHasWeight e n}
```

`DistanceMinima`: sourceQuantumHasWeight

Complete checked declaration at the immutable proof commit.

```
def sourceQuantumHasWeight (e : Fin 3 → Fin 4 → ZMod L) (n : ℕ∞) : Prop :=
  ∃ x z : Qubit L → ZMod 2,
    (∀ c,(∑ q,GeneralLP.HZ e c q*x q)=0) ∧
    (∀ c,(∑ q,GeneralLP.HX e c q*z q)=0) ∧
    ¬(fullSpanX e x ∧ fullSpanZ e z) ∧
    n=(((CoverWeight.support x)∪(CoverWeight.support z)).card : ℕ∞)
```

`DistanceMinima`: sourceQuantumDistance

Complete checked declaration at the immutable proof commit.

```
noncomputable def sourceQuantumDistance (e : Fin 3 → Fin 4 → ZMod L) : ℕ∞ :=
  sInf {n | sourceQuantumHasWeight e n}
```

D Actual incidence and the three profiles

The source kernel is the input of the count classification. The abstract normal graph and local edge bounds are proved in its imports, rather than assumed as new hypotheses.

`ActualIncidence`: source_kernel_zero_or_two

Complete checked declaration at the immutable proof commit.

```
theorem source_kernel_zero_or_two (e : Fin 3 → Fin 4 → ZMod L)
    (h4 : NoFour (CEdge e)) (h6 : NoSix (CEdge e))
    (x : Qubit L → ZMod 2)
    (hk : ∀ c, (∑ q,GeneralLP.HZ e c q*x q)=0)
    (hw : (CoverWeight.support x).card≤11) (c : SourceModel.ZCheck L) :
    ((CoverWeight.support x).filter (fun q => ZEdge e q c)).card=0 ∨
      ((CoverWeight.support x).filter (fun q => ZEdge e q c)).card=2 := by
  have hv (c : SourceModel.ZCheck L) (q : Qubit L) :
      GeneralLP.HZ e c q = if ZEdge e q c then 1 else 0 := by
    by_cases h : ZEdge e q c
    · simp only [h,if_true]
      exact CoverWeight.nonzero_one _ ((quantum_incidence e q c).mpr h)
    · simp only [h,if_false]
      exact not_ne_iff.mp (fun hn => h ((quantum_incidence e q c).mp hn))
  apply LowWeightChecks.kernel_check_zero_or_two (ZEdge e)
    (quantum_no_four e h4) (quantum_no_six e h6) (quantum_three_checks e) x ?_ hw c
  intro d
  simpa only [hv,ite_mul,one_mul,zero_mul] using hk d
```

`SourceProfiles`: source_mixed_count_profiles

Complete checked declaration at the immutable proof commit.

```
theorem source_mixed_count_profiles (e : Fin 3 → Fin 4 → ZMod L)
    (h4 : NoFour (CEdge e)) (h6 : NoSix (CEdge e))
    (u : Fin 4 → Fin 4 → ZMod L → ZMod 2)
    (v : Fin 3 → Fin 3 → ZMod L → ZMod 2)
    (hk : ∀ c, (∑ q,GeneralLP.HZ e c q*GeneralLP.join u v q)=0)
    (hw : (CoverWeight.support (GeneralLP.join u v)).card≤11)
    (hU : 0<∑ j,leftCount u j) (hV : 0<∑ i,rightCount v i) :
    (sortedCounts (leftCount u),sortedCounts (rightCount v))=([1,1,1,1],[1,1,1]) ∨
    (sortedCounts (leftCount u),sortedCounts (rightCount v))=([2,2,2,0],[2,2,0]) ∨
    (sortedCounts (leftCount u),sortedCounts (rightCount v))=([5,1,1,1],[1,1,1]) := by
  have htotal : (∑ j,leftCount u j)+(∑ i,rightCount v i)≤11 := by
    simpa only [support_weight_counts] using hw
  have hsy : ∀ j i t, GeneralLP.Syndrome e u v j i t=0 := by
    intro j i t
    exact (GeneralLP.z_action e u v j i t).symm.trans (hk ((j,i),t))
  obtain ⟨p,hp,hap,hbp⟩ := common_count_parity e u v hsy
  have hboundA (x : ℕ) (hx : x∈sortedCounts (leftCount u)) : x≤11 := by
    obtain ⟨j,rfl⟩ := (sortedCounts_mem _ _).mp hx
    have hs : leftCount u j≤∑ k,leftCount u k :=
      Finset.single_le_sum (fun k _ => Nat.zero_le _) (Finset.mem_univ j)
    omega
  have hboundB (x : ℕ) (hx : x∈sortedCounts (rightCount v)) : x≤11 := by
    obtain ⟨i,rfl⟩ := (sortedCounts_mem _ _).mp hx
    have hs : rightCount v i≤∑ k,rightCount v k :=
      Finset.single_le_sum (fun k _ => Nat.zero_le _) (Finset.mem_univ i)
    omega
```

```
  have hparA (x : ℕ) (hx : x∈sortedCounts (leftCount u)) : x%2=p := by
    obtain ⟨j,rfl⟩ := (sortedCounts_mem _ _).mp hx
    exact hap j
  have hparB (x : ℕ) (hx : x∈sortedCounts (rightCount v)) : x%2=p := by
    obtain ⟨i,rfl⟩ := (sortedCounts_mem _ _).mp hx
    exact hbp i
  have hm := mem_countCases (sortedCounts (leftCount u)) (sortedCounts (rightCount v)) p hp
    (sortedCounts_length _) (sortedCounts_length _) (sortedCounts_pairwise _)
    (sortedCounts_pairwise _) hboundA hboundB hparA hparB
    (by simpa only [sortedCounts_sum] using hU)
    (by simpa only [sortedCounts_sum] using hV)
    (by simpa only [sortedCounts_sum] using htotal)
  apply classified_of_mem_countCases _ hm
  apply Bool.and_eq_true_iff.mpr
  constructor
  · apply List.all_eq_true.mpr
    intro a ha
    obtain ⟨j,rfl⟩ := (sortedCounts_mem _ _).mp ha
    apply decide_eq_true_iff.mpr
    simpa only [sortedCounts_map_sum,pow_two] using source_left_local_mantel e h4 h6 u v hk hw j
  · apply List.all_eq_true.mpr
    intro b hb
    obtain ⟨i,rfl⟩ := (sortedCounts_mem _ _).mp hb
    apply decide_eq_true_iff.mpr
    simpa only [sortedCounts_map_sum,pow_two] using source_right_local_mantel e h4 h6 u v hk hw i
```

E Constant rows and canonical support encodings

The following complete declarations show the actual support-to-catalog interfaces, including the exceptional constant-label branch.

`ConstantRows`: constant_singleton_rows

Complete checked declaration at the immutable proof commit.

```
theorem constant_singleton_rows {J : Type*} [Nonempty J]
    (e : Fin 3 → Fin 4 → ZMod L)
    (u : Fin 4 → Fin 4 → ZMod L → ZMod 2)
    (v : Fin 3 → Fin 3 → ZMod L → ZMod 2)
    (hk : ∀ c,(∑ q,GeneralLP.HZ e c q*GeneralLP.join u v q)=0)
    (a : J → Fin 4) (K : Fin 4) (H : Fin 3)
    (s : J → ZMod L) (t : Fin 3 → ZMod L)
    (hu : ∀ j k' s',u (a j) k' s'=if k'=K ∧ s'=s j then 1 else 0)
    (hv : ∀ i l' t',v l' i t'=if l'=H ∧ t'=t i then 1 else 0) :
    ∃ z,(∀ j,u (a j)=rowU e H K z (a j)) ∧ v=rowV e H K z := by
  have hc (j : J) (i : Fin 3) : s j+e i K=t i-e H (a j) :=
    singleton_source_check_matches e u v hk (a j) K i H (s j) (t i) (hu j) (hv i)
  obtain ⟨z,hs,ht⟩ := constant_label_fibers e a (fun i : Fin 3 => i) K H s t hc
  refine ⟨z,?_,?_⟩
  · intro j
    funext k' s'
    rw [hu,rowU_indicator,hs j]
  · funext l' i t'
    rw [hv,rowV_indicator,ht i]
```

`SourceSeven`: source_seven_singleton_profile_stabilizer

Complete checked declaration at the immutable proof commit.

```
theorem source_seven_singleton_profile_stabilizer
    (e : Fin 3 → Fin 4 → ZMod L)
    (h4 : TannerSix.NoFour (CEdge e)) (h6 : TannerSix.NoSix (CEdge e))
    (hR0 : ∀ j,e 0 j=0) (hC0 : ∀ i,e i 0=0)
    (hCP : NoColumnPartition e) (hP : NoTripleRow e)
    (u : Fin 4 → Fin 4 → ZMod L → ZMod 2)
    (v : Fin 3 → Fin 3 → ZMod L → ZMod 2)
    (hker : ∀ c,(∑ q,GeneralLP.HZ e c q*GeneralLP.join u v q)=0)
    (hU : ∀ j,leftCount u j=1) (hV : ∀ i,rightCount v i=1) :
    GeneralLP.InSpanX e u v := by
  obtain ⟨k,s,l,t,hu,hv,hc,hcase⟩ := actual_singleton_label_alternative e h4 u v hker
    (fun j : Fin 4 => j) Function.injective_id hU hV
  rcases hcase with ⟨K,H,hk,hl⟩|⟨hk,hl⟩
  · apply constant_all_singletons_stabilizer e u v hker K H s t
    · simpa only [hk] using hu
    · simpa only [hl] using hv
  · obtain ⟨p₄,hp₄⟩ := permutation4_complete k hk
    obtain ⟨p₃,hp₃⟩ := permutation3_complete l hl
    have hp : 6*p₄.val+p₃.val<144 := by omega
    obtain ⟨c,hcert,hn⟩ := seven_certificates_complete ⟨_,hp⟩
    have hD : ClassicalAtLeast e 0 := by intro x hx hx0;exact Nat.zero_le _
    exact False.elim (certified_pattern_has_no_source_realization e h4 h6 hR0 hC0 hCP hP
      0 hD 7 _ c hcert (fun h => False.elim (hn h))
      ⟨seven_realization_of_labels e k l s t hc p₄ p₃ hp₄ hp₃⟩)
```

TenGeometry: source_ten_geometry

Complete checked declaration at the immutable proof commit.

```
theorem source_ten_geometry (e : Fin 3 → Fin 4 → ZMod L)
    (h4 : TannerSix.NoFour (CEdge e))
    (hR0 : ∀ k,e 0 k=0) (hC0 : ∀ l,e l 0=0)
    (u : Fin 4 → Fin 4 → ZMod L → ZMod 2)
    (v : Fin 3 → Fin 3 → ZMod L → ZMod 2)
    (hk : ∀ c,(∑ q,GeneralLP.HZ e c q*GeneralLP.join u v q)=0)
    (hU0 : leftCount u 0=0) (hV0 : rightCount v 0=0)
    (hU : ∀ j : Fin 3,leftCount u j.succ=2)
    (hV : ∀ i : Fin 2,rightCount v i.succ=2) : Nonempty (TenGeometry e u v) := by
  classical
  have hu0 : ∀ k s,u 0 k s=0 := by
    have h := zero_of_support_card_zero (fun q : Fin 4 × ZMod L => u 0 q.1 q.2) hU0
    intro k s
    exact congrFun h (k,s)
  have hv0 : ∀ l s,v l 0 s=0 := by
    have h := zero_of_support_card_zero (fun q : Fin 3 × ZMod L => v q.1 0 q.2) hV0
    intro l s
    exact congrFun h (l,s)
  have hleft (j : Fin 3) : ∃ a b s,a<b ∧
      ∀ k z,u j.succ k z=if (k=a ∨ k=b) ∧ z=s then 1 else 0 := by
    obtain ⟨a,b,s,hab,hf⟩ := left_pair_same_fiber e hR0 u v hk hv0 j.succ (hU j)
    rcases lt_or_gt_of_ne hab with h|h
    · exact ⟨a,b,s,h,hf⟩
    · exact ⟨b,a,s,h,by simpa only [or_comm] using hf⟩
  have hright (i : Fin 2) : ∃ a b s,a<b ∧
      ∀ l z,v l i.succ z=if (l=a ∨ l=b) ∧ z=s then 1 else 0 := by
    obtain ⟨a,b,s,hab,hf⟩ := right_pair_same_fiber e hC0 u v hk hu0 i.succ (hV i)
    rcases lt_or_gt_of_ne hab with h|h
    · exact ⟨a,b,s,h,hf⟩
    · exact ⟨b,a,s,h,by simpa only [or_comm] using hf⟩
  choose a b S hab hu using hleft
  choose l m T hlm hv using hright
  have hm (i : Fin 2) (j : Fin 3) : ∃ f : Fin 2,∀ t : Fin 2,
      S j+e i.succ ((![a j,b j]) t)=
        T i-e ((![l i,m i]) (if t=f then 0 else 1)) j.succ := by
    have hcase := source_pair_cross_matching e h4 hR0 hC0 u v hk j.succ (by simp)
      i.succ (by simp) (a j) (b j) (ne_of_lt (hab j)) (l i) (m i) (ne_of_lt (hlm i))
      (S j) (T i) (hu j) (hv i)
    rcases hcase with ⟨h0,h1⟩|⟨h0,h1⟩
    · refine ⟨0,?_⟩
      intro t
      fin_cases t
      · simpa using h0
      · simpa using h1
    · refine ⟨1,?_⟩
      intro t
      fin_cases t
      · simpa using h0
      · simpa using h1
  choose flip hf using hm
  exact ⟨{
    K := fun j => ![a j,b j]
    H := fun i => ![l i,m i]
    S := S
    T := T
    orderedK := by simpa using hab
    orderedH := by simpa using hlm
```

```
    left_repr := by simpa using hu
    right_repr := by simpa using hv
    flip := flip
    matching := hf
  ⟩
```

TenEncoding: ten_geometry_encodes

Complete checked declaration at the immutable proof commit.

```
theorem ten_geometry_encodes
    (e : Fin 3 → Fin 4 → ZMod L)
    (hR0 : ∀ k,e 0 k=0) (hC0 : ∀ l,e l 0=0)
    (u : Fin 4 → Fin 4 → ZMod L → ZMod 2)
    (v : Fin 3 → Fin 3 → ZMod L → ZMod 2)
    (d : TenGeometry e u v) :
    ∃ p : Fin 124416,Nonempty (PatternRealization e 10 p.val) := by
  classical
  choose c hc using (fun j => ordered_column_pair_complete (d.K j 0) (d.K j 1) (d.orderedK j))
  choose r hr using (fun i => ordered_row_pair_complete (d.H i 0) (d.H i 1) (d.orderedH i))
  have hK (j : Fin 3) (a : Fin 2) : columnPairValue (c j).val a.val=(d.K j a).val := by
    fin_cases a
    · exact (hc j).1
    · exact (hc j).2
  have hH (i : Fin 2) (a : Fin 2) : rowPairValue (r i).val a.val=(d.H i a).val := by
    fin_cases a
    · exact (hr i).1
    · exact (hr i).2
  let flips : Fin 6 → Fin 2 := ![d.flip 0 0,d.flip 0 1,d.flip 0 2,d.flip 1 0,d.flip 1 1,d.flip 1 2]
  obtain ⟨bits,hbits⟩ := six_binary_flips_complete flips
  have hflip (i : Fin 2) (j : Fin 3) : flip10 (tenRank c r bits) i.val j.val=(d.flip i j).val := by
    rw [tenRank_flip]
    fin_cases i <;> fin_cases j
    · simpa [flips] using hbits 0
    · simpa [flips] using hbits 1
    · simpa [flips] using hbits 2
    · simpa [flips] using hbits 3
    · simpa [flips] using hbits 4
    · simpa [flips] using hbits 5
  exact ⟨⟨tenRank c r bits,tenRank_bound c r bits⟩,
    ⟨ten_realization_of_geometry d hR0 hC0 c r bits hK hH hflip⟩⟩
```

TenEncoding: actual_canonical_ten_support_encodes

Complete checked declaration at the immutable proof commit.

```
theorem actual_canonical_ten_support_encodes
    (e : Fin 3 → Fin 4 → ZMod L)
    (h4 : TannerSix.NoFour (CEdge e))
    (hR0 : ∀ k,e 0 k=0) (hC0 : ∀ l,e l 0=0)
    (u : Fin 4 → Fin 4 → ZMod L → ZMod 2)
    (v : Fin 3 → Fin 3 → ZMod L → ZMod 2)
    (hk : ∀ c,(∑ q,GeneralLP.HZ e c q*GeneralLP.join u v q)=0)
    (hU0 : leftCount u 0=0) (hV0 : rightCount v 0=0)
    (hU : ∀ j : Fin 3,leftCount u j.succ=2)
    (hV : ∀ i : Fin 2,rightCount v i.succ=2) :
    ∃ p : Fin 124416,Nonempty (PatternRealization e 10 p.val) := by
  obtain ⟨d⟩ := source_ten_geometry e h4 hR0 hC0 u v hk hU0 hV0 hU hV
```

```
  exact ten_geometry_encodes e hR0 hC0 u v d
```

`K23Numbering`: actual_k23_numbering

Complete checked declaration at the immutable proof commit.

```
theorem actual_k23_numbering (e : Fin 3 → Fin 4 → ZMod L)
    (h4 : NoFour (CEdge e)) (h6 : NoSix (CEdge e))
    (u : Fin 4 → Fin 4 → ZMod L → ZMod 2)
    (v : Fin 3 → Fin 3 → ZMod L → ZMod 2)
    (hk : ∀ c,(∑ q,GeneralLP.HZ e c q*GeneralLP.join u v q)=0)
    (hw : (CoverWeight.support (GeneralLP.join u v)).card≤11)
    (hU : leftCount u 3=5) (hV : ∀ i,rightCount v i=1) : Nonempty (K23Numbering e u) := by
  classical
  obtain ⟨f⟩ := exceptional_group_is_k23 e h4 h6 u v hk hw 3 hU hV
  have hcoords (x : Fin 5) : ∃ k s,(f.symm x).val=.inl ((3,k),s) ∧ u 3 k s≠0 :=
    left_support_coordinates u 3 (f.symm x).val (f.symm x).property
  choose K S hq using hcoords
  refine ⟨{
    K := K
    S := S
    nonzero := fun x => (hq x).2
    injective := ?_
    covers := ?_
    adjacency := ?_
  }⟩
  · intro x y he
    apply f.symm.injective
    apply Subtype.ext
    rw [(hq x).1,(hq y).1]
    have ha := congrArg Prod.fst he
    have hb := congrArg Prod.snd he
    change K x=K y at ha
    change S x=S y at hb
    rw [ha,hb]
  · intro k s hn
    let z : leftSupport u 3 := ⟨.inl ((3,k),s),by simp [hn]⟩
    refine ⟨f z,?_⟩
    have he := (hq (f z)).1
    simp only [f.symm_apply_apply] at he
    change Sum.inl ((3,k),s)=Sum.inl ((3,K (f z)),S (f z)) at he
    have hp := Sum.inl.inj he
    exact ⟨(congrArg (fun p => p.1.2) hp).symm,(congrArg Prod.snd hp).symm⟩
  · intro x y
    have hx : (f.symm x).val=.inl ((3,K x),S x) := (hq x).1
    have hy : (f.symm y).val=.inl ((3,K y),S y) := (hq y).1
    have hn : f.symm x≠f.symm y ↔ x≠y := f.symm.injective.ne_iff
    change (x≠y ∧ ∃ c,ZEdge e (.inl ((3,K x),S x)) c ∧ ZEdge e (.inl ((3,K y),S y)) c) ↔
      (SimpleGraph.turanGraph 5 2).Adj x y
    rw [← f.symm.map_adj_iff]
    change (x≠y ∧ _) ↔ (f.symm x≠f.symm y ∧ _)
    rw [hn,hx,hy]
```

`ElevenGeometry`: actual_eleven_geometry

Complete checked declaration at the immutable proof commit.

```
theorem actual_eleven_geometry (e : Fin 3 → Fin 4 → ZMod L)
```

```
    (h4 : NoFour (CEdge e)) (h6 : NoSix (CEdge e))
    (u : Fin 4 → Fin 4 → ZMod L → ZMod 2)
    (v : Fin 3 → Fin 3 → ZMod L → ZMod 2)
    (hk : ∀ c,(∑ q,GeneralLP.HZ e c q*GeneralLP.join u v q)=0)
    (hw : (CoverWeight.support (GeneralLP.join u v)).card≤11)
    (n : K23Numbering e u) (c : K23InternalChecks e u n)
    (K : Fin 3 → Fin 4) (S : Fin 3 → ZMod L) (H : Fin 3 → Fin 3) (T : Fin 3 → ZMod L)
    (hK : Function.Injective K) (hH : Function.Injective H)
    (hMatch : ∀ j i,S j+e i (K j)=T i-e (H i) (lowColumn j))
    (hv : ∀ i l t,v l i t=if l=H i ∧ t=T i then 1 else 0) : Nonempty (ElevenGeometry e) := by
  obtain ⟨o,ho⟩ := k23_has_orientation c
  refine ⟨{
    singleK := K
    singleS := S
    rightH := H
    rightT := T
    singleK_injective := hK
    rightH_injective := hH
    centerK := fun a => n.K (k23Center a)
    centerS := fun a => n.S (k23Center a)
    leafK := fun i => n.K (k23Leaf (leafByRow c i))
    leafS := fun i => n.S (k23Leaf (leafByRow c i))
    orientation := o
    adjacent_labels := ?_
    single_matching := hMatch
    internal_matching := ?_
    leaf_matching := actual_leaf_cross_matches e h4 h6 u v hk hw n c H T hv
  }⟩
  · intro a i
    exact (adjacent_numbered_labels_distinct e u n (k23Center a) (k23Leaf (leafByRow c i))
      (k23_center_ne_leaf a _) (c.check a (leafByRow c i))
      (c.center_edge a _) (c.leaf_edge a _)).symm
  · intro a i
    have h1 := (c.center_edge a (leafByRow c i)).2
    have h2 := (c.leaf_edge a (leafByRow c i)).2
    change (c.check a (leafByRow c i)).2=n.S (k23Center a)+
      e (c.check a (leafByRow c i)).1.2 (n.K (k23Center a)) at h1
    change (c.check a (leafByRow c i)).2=n.S (k23Leaf (leafByRow c i))+
      e (c.check a (leafByRow c i)).1.2 (n.K (k23Leaf (leafByRow c i))) at h2
    rw [ho a i] at h1 h2
    exact h1.symm.trans h2
```

`SourceEleven`: source_canonical_eleven_stabilizer

Complete checked declaration at the immutable proof commit.

```
theorem source_canonical_eleven_stabilizer
    (e : Fin 3 → Fin 4 → ZMod L)
    (h4 : NoFour (CEdge e)) (h6 : NoSix (CEdge e))
    (hR0 : ∀ j,e 0 j=0) (hC0 : ∀ i,e i 0=0)
    (hCP : NoColumnPartition e) (hP : NoTripleRow e)
    (D : ℕ) (hD : 10<D) (hC : ClassicalAtLeast e D)
    (u : Fin 4 → Fin 4 → ZMod L → ZMod 2)
    (v : Fin 3 → Fin 3 → ZMod L → ZMod 2)
    (hk : ∀ c,(∑ q,GeneralLP.HZ e c q*GeneralLP.join u v q)=0)
    (hw : (CoverWeight.support (GeneralLP.join u v)).card≤11)
    (hU : ∀ j : Fin 3,leftCount u (lowColumn j)=1)
    (hU3 : leftCount u 3=5) (hV : ∀ i,rightCount v i=1) :
    GeneralLP.InSpanX e u v := by
```

```
obtain ⟨K,S,H,T,hu,hv,hmatch,hcase⟩ := actual_singleton_label_alternative e h4 u v hk
  lowColumn lowColumn_injective hU hV
rcases hcase with ⟨K₀,H₀,hK,hH⟩|⟨hK,hH⟩
· have hu' : ∀ j k s,u (lowColumn j) k s=if k=K₀ ∧ s=S j then 1 else 0 := by
    simpa only [hK] using hu
  have hv' : ∀ i l t,v l i t=if l=H₀ ∧ t=T i then 1 else 0 := by
    simpa only [hH] using hv
  obtain ⟨z,hleft,hright⟩ := constant_singleton_rows e u v hk lowColumn K₀ H₀ S T hu' hv'
  apply single_exception_cancellation e D (by omega) hC u (rowU e H₀ K₀ z)
    v (rowV e H₀ K₀ z) hk (source_row_full_span e H₀ K₀ z) 3 (by omega)
    (by rw [source_row_left_count]) _ hright
  intro j hj
  have hlt : j.val<3 := by
    have hbound := j.isLt
    have hn : j.val≠3 := fun he => hj (Fin.ext he)
    omega
  let j' : Fin 3 := ⟨j.val,hlt⟩
  have he : lowColumn j'=j := Fin.ext rfl
  simpa only [he] using hleft j'
· obtain ⟨n⟩ := actual_k23_numbering e h4 h6 u v hk hw hU3 hV
  obtain ⟨c⟩ := actual_k23_internal_checks e h4 h6 u v hk hw n
  obtain ⟨d⟩ := actual_eleven_geometry e h4 h6 u v hk hw n c K S H T hK hH hmatch hv
  obtain ⟨p,hp⟩ := eleven_geometry_encodes e d
  exact False.elim (no_eleven_pattern_realization e h4 h6 hR0 hC0 hCP hP D hD hC p hp)
```

F Integer certificate interpretation and cyclic torsion

Integer identities are interpreted in every additive group. The finite catalog supplies witnesses to this checked implication; it supplies no additional axiom.

`IntegerSoundness`: integer_certificate_sound

Complete checked declaration at the immutable proof commit.

```
theorem integer_certificate_sound {m n : ℕ} {A : Type*} [AddCommGroup A]
    (M : Fin m → Fin n → ℤ) (a : Fin m → ℤ) (target : Fin n → ℤ)
    (x : Fin n → A) (hrows : ∀ i, integerDot (M i) x=0)
    (hcert : ∀ j, target j=∑ i, a i*M i j) : integerDot target x=0 := by
  change (∑ j, target j • x j)=0
  calc
    _ = ∑ j, ∑ i, (a i*M i j) • x j := by
      simp_rw [hcert,Finset.sum_smul]
    _ = ∑ i, a i • integerDot (M i) x := by
      rw [Finset.sum_comm]
      simp only [integerDot,Finset.smul_sum,mul_zsmul]
    _ = 0 := by simp [hrows]
```

`KernelCatalog`: checked_fast_equations

Complete checked declaration at the immutable proof commit.

```
theorem checked_fast_equations {A : Type*} [AddCommGroup A]
    (w p : Nat) (c : Certificate) (hc : checkCertificateFast w p c=true)
    (x : Fin (w+6) → A)
    (hr : ∀ i : Fin 18, TruncatedDistance.integerDot (sourceRow w p i) x=0) :
    ∀ t : Fin (targetArity c.kind),
      TruncatedDistance.integerDot (certificateTarget w c t) x=0 := by
  have hcert (t : Fin (targetArity c.kind)) (j : Fin (w+6)) :
      certificateTarget w c t j=∑ i : Fin 18,c.coeff t.val i.val*sourceRow w p i j := by
    have ht := List.all_eq_true.mp ((Bool.and_eq_true_iff.mp hc).2)
      t.val (List.mem_range.mpr t.isLt)
    have hj := List.all_eq_true.mp ht j.val (List.mem_range.mpr j.isLt)
    have heq := of_decide_eq_true hj
    simpa only [certificateTarget,certificateTargetNat,dotNat18_eq_sum,
      sparseProduct_eq,sourceRowNat_eq] using heq
  intro t
  exact TruncatedDistance.integer_certificate_sound (sourceRow w p)
    (fun i => c.coeff t.val i.val) (certificateTarget w c t) x hr (hcert t)
```

`PatternRealizations`: PatternRealization

Complete checked declaration at the immutable proof commit.

```
structure PatternRealization (e : Fin 3 → Fin 4 → ZMod L) (w p : ℕ) where
  qubits : Fin w → Qubit L
  checks : (r : Fin 18) → r.val<edgeCount w → SourceModel.ZCheck L
  left_bound : ∀ r : Fin 18,∀ hr : r.val<edgeCount w,(edge w p r.val).a<w
  right_bound : ∀ r : Fin 18,∀ hr : r.val<edgeCount w,(edge w p r.val).b<w
  labels : ∀ v : Fin w,VertexLabel (vertex w p v.val) (qubits v)
  check_column : ∀ r : Fin 18,∀ hr : r.val<edgeCount w,(edge w p r.val).j=(checks r hr).1.1.val
  check_row : ∀ r : Fin 18,∀ hr : r.val<edgeCount w,(edge w p r.val).i=(checks r hr).1.2.val
  left_edge : ∀ r : Fin 18,∀ hr : r.val<edgeCount w,
```

```
    ZEdge e (qubits ⟨(edge w p r.val).a,left_bound r hr⟩) (checks r hr)
  right_edge : ∀ r : Fin 18,∀ hr : r.val<edgeCount w,
    ZEdge e (qubits ⟨(edge w p r.val).b,right_bound r hr⟩) (checks r hr)
```

`PatternRealizations`: certified_pattern_has_no_source_realization

Complete checked declaration at the immutable proof commit.

```
theorem certified_pattern_has_no_source_realization
    (e : Fin 3 → Fin 4 → ZMod L)
    (h4 : TannerSix.NoFour (CEdge e)) (h6 : TannerSix.NoSix (CEdge e))
    (hR0 : ∀ j,e 0 j=0) (hC0 : ∀ i,e i 0=0)
    (hCP : NoColumnPartition e) (hP : NoTripleRow e)
    (D : ℕ) (hC : ClassicalAtLeast e D) (w p : ℕ) (c : Certificate)
    (hc : checkCertificateFast w p c=true)
    (hExtra : needsClassicalBound c.kind → 10<D) : ¬Nonempty (PatternRealization e w p) := by
  rintro ⟨h⟩
  have hkind : c.kind<64 := of_decide_eq_true
    ((Bool.and_eq_true_iff.mp ((Bool.and_eq_true_iff.mp hc).1)).2)
  have hz := checked_fast_equations w p c hc (realizationVector e w h.qubits)
    (realization_rows_zero e hR0 hC0 w p h)
  apply source_target_obstruction e h4 h6 hR0 hC0 hCP hP D hC ⟨c.kind,hkind⟩ hExtra
  intro t
  have ht := hz t
  change integerDot (extendSix w (targetVector c.kind t.val)) (realizationVector e w h.qubits)=0 at ht
  rw [dot_extendSix,realization_prefix] at ht
  simpa only [targetForm,sourceVars,sourceExponents] using ht
```

`TorsionClassical`: torsionWord

Complete checked declaration at the immutable proof commit.

```
def torsionWord (n : ℕ) (c : Fin 4) (q : Fin 4 × ZMod L) : ZMod 2 :=
  (if q.1 = 0 then torsionBit n q.2 else 0) +
  (if q.1 = c then torsionBit n q.2 else 0)
```

`TorsionClassical`: source_classical_word_le10

Complete checked declaration at the immutable proof commit.

```
theorem source_classical_word_le10 (e : Fin 3 → Fin 4 → ZMod L)
    (c : Fin 4) (hc : c ≠ 0) (h0 : ∀ i, e i 0=0)
    (ht : ∀ i, 5 • e i c=0) :
    ∃ x : (Fin 4 × ZMod L) → ZMod 2,
      (∀ a, (∑ q, GeneralLP.C e a q*x q)=0) ∧ x≠0 ∧
      (CoverWeight.support x).card≤10 := by
  refine ⟨torsionWord 5 c,torsionWord_source_kernel e 5 c h0 ht,
    torsionWord_nonzero 5 c hc,?_⟩
  simpa using torsionWord_weight_le (L := L) 5 (by decide) c hc
```

`CoverageCore`: Covered

Complete checked declaration at the immutable proof commit.

```
def Covered (w start count : Nat) : Prop :=
  ∀ p : Fin count,∃ c : Certificate,checkCertificateFast w (start+p.val) c=true
```

`CoverageCore`: covered_single

Complete checked declaration at the immutable proof commit.

```
theorem covered_single (w start : Nat) (c : Certificate)
    (h : checkCertificateFast w start c=true) : Covered w start 1 := by
  intro p
  have hp : p.val=0 := by omega
  refine ⟨c,?_⟩
  simpa only [hp,Nat.add_zero] using h
```

`CoverageCore`: covered_append

Complete checked declaration at the immutable proof commit.

```
theorem covered_append (w start a b : Nat)
    (ha : Covered w start a) (hb : Covered w (start+a) b) : Covered w start (a+b) := by
  intro p
  by_cases h : p.val<a
  · exact ha ⟨p.val,h⟩
  · have ht : p.val-a<b := by have := p.isLt;omega
    obtain ⟨c,hc⟩ := hb ⟨p.val-a,ht⟩
    refine ⟨c,?_⟩
    have he : start+a+(p.val-a)=start+p.val := by omega
    simpa only [he] using hc
```

G Transport and the complete global reduction

Canonical group placement is discharged by permutation and re-normalization. The global proof below includes all three profiles and the pure sectors.

`ProfileTransport`: DistanceCriterion

Complete checked declaration at the immutable proof commit.

```
structure DistanceCriterion (e : Fin 3 → Fin 4 → ZMod L) (D : ℕ) : Prop where
  no_four : NoFour (CEdge e)
  no_six : NoSix (CEdge e)
  row_zero : ∀ j,e 0 j=0
  column_zero : ∀ i,e i 0=0
  no_column_partition : NoColumnPartition e
  no_triple_row : NoTripleRow e
  classical_bound : ClassicalAtLeast e D
  transpose_bound : TransposeAtLeast e D
```

`ProfileTransport`: normPerm_criterion

Complete checked declaration at the immutable proof commit.

```
theorem normPerm_criterion (e : Fin 3 → Fin 4 → ZMod L) (D : ℕ)
    (h : DistanceCriterion e D) (r : Equiv.Perm (Fin 3)) (c : Equiv.Perm (Fin 4)) :
    DistanceCriterion (normPermE e r c) D := by
  have ht := renormalized_entry_tests e h.row_zero h.no_column_partition h.no_triple_row r c
  exact {
    no_four := normalization_no_four _ 0 0 (perm_no_four e r c h.no_four)
    no_six := normalization_no_six _ 0 0 (perm_no_six e r c h.no_six)
    row_zero := fun j => normalizeAt_row _ 0 0 j
    column_zero := fun i => normalizeAt_column _ 0 0 i
    no_column_partition := ht.1
    no_triple_row := ht.2
    classical_bound := normalization_classical_bound _ 0 0 D (permutation_classical_bound e r c D h.
    classical_bound)
    transpose_bound := normalization_transpose_bound _ 0 0 D (permutation_transpose_bound e r c D h.
    transpose_bound)
  }
```

`ProfileTransport`: normPerm_span

Complete checked declaration at the immutable proof commit.

```
theorem normPerm_span (e : Fin 3 → Fin 4 → ZMod L)
    (r : Equiv.Perm (Fin 3)) (c : Equiv.Perm (Fin 4))
    (u : Fin 4 → Fin 4 → ZMod L → ZMod 2)
    (v : Fin 3 → Fin 3 → ZMod L → ZMod 2) :
    GeneralLP.InSpanX (normPermE e r c) (normPermU e r c u) (normPermV e r c v) ↔
      GeneralLP.InSpanX e u v := by
  unfold normPermE normPermU normPermV
  rw [normalize_is_gauge,full_span_gauge_iff]
  exact perm_source_span_iff e r c u v
```

`TwelveConditional`: source_kernel_below_twelve_of_coverage

Complete checked declaration at the immutable proof commit.

```
theorem source_kernel_below_twelve_of_coverage
    (hCoverage : Covered 10 0 124416)
    (e : Fin 3 → Fin 4 → ZMod L) (D : ℕ) (h : DistanceCriterion e D)
    (u : Fin 4 → Fin 4 → ZMod L → ZMod 2)
    (v : Fin 3 → Fin 3 → ZMod L → ZMod 2)
    (hk : ∀ a,(∑ q,GeneralLP.HZ e a q*GeneralLP.join u v q)=0)
    (hw : (CoverWeight.support (GeneralLP.join u v)).card<min D 12) :
    GeneralLP.InSpanX e u v := by
  have hwD := (lt_min_iff.mp hw).1
  have hw12 := (lt_min_iff.mp hw).2
  by_cases hU : 0<∑ j,leftCount u j
  · by_cases hV : 0<∑ i,rightCount v i
    · rcases source_mixed_count_profiles e h.no_four h.no_six u v hk (by omega) hU hV with h7|h10|h11
      · apply source_seven_singleton_profile_stabilizer e h.no_four h.no_six h.row_zero h.column_zero
          h.no_column_partition h.no_triple_row u v hk
        · intro j
          have hm := (sortedCounts_mem (leftCount u) (leftCount u j)).mpr ⟨j,rfl⟩
          have he : sortedCounts (leftCount u)=[1,1,1,1] := congrArg Prod.fst h7
          rw [he] at hm
          simpa using hm
        · intro i
          have hm := (sortedCounts_mem (rightCount v) (rightCount v i)).mpr ⟨i,rfl⟩
          have he : sortedCounts (rightCount v)=[1,1,1] := congrArg Prod.snd h7
          rw [he] at hm
          simpa using hm
      · have ha : sortedCounts (leftCount u)=[2,2,2,0] := congrArg Prod.fst h10
        have hb : sortedCounts (rightCount v)=[2,2,0] := congrArg Prod.snd h10
        have hweight : (CoverWeight.support (GeneralLP.join u v)).card=10 := by
          rw [support_weight_counts,← sortedCounts_sum (leftCount u),← sortedCounts_sum (rightCount v)
    ,ha,hb]
          rfl
        have hD : 10<D := by omega
        obtain ⟨j₀,hj₀,hjrest⟩ := ten_left_marker (leftCount u) ha
        obtain ⟨i₀,hi₀,hirest⟩ := ten_right_marker (rightCount v) hb
        let r : Equiv.Perm (Fin 3) := Equiv.swap 0 i₀
        let c : Equiv.Perm (Fin 4) := Equiv.swap 0 j₀
        have hr0 : r 0=i₀ := Equiv.swap_apply_left _ _
        have hc0 : c 0=j₀ := Equiv.swap_apply_left _ _
        let e' := normPermE e r c
        let u' := normPermU e r c u
        let v' := normPermV e r c v
        have h' : DistanceCriterion e' D := normPerm_criterion e D h r c
        have hk' := normPerm_kernel e r c u v hk
        have hU0' : leftCount u' 0=0 := by
          change leftCount (normPermU e r c u) 0=0
          rw [normPerm_left_count,hc0]
          exact hj₀
        have hV0' : rightCount v' 0=0 := by
          change rightCount (normPermV e r c v) 0=0
          rw [normPerm_right_count,hr0]
          exact hi₀
        have hU' (j : Fin 3) : leftCount u' j.succ=2 := by
          change leftCount (normPermU e r c u) j.succ=2
          rw [normPerm_left_count]
          apply hjrest
          intro he
          have hj : j.succ=0 := c.injective (he.trans hc0.symm)
```

```
      simpa using hj
    have hV' (i : Fin 2) : rightCount v' i.succ=2 := by
      change rightCount (normPermV e r c v) i.succ=2
      rw [normPerm_right_count]
      apply hirest
      intro he
      have hi : i.succ=0 := r.injective (he.trans hr0.symm)
      simpa using hi
    exact False.elim (source_canonical_ten_impossible_of_coverage hCoverage e'
      h'.no_four h'.no_six h'.row_zero h'.column_zero h'.no_column_partition h'.no_triple_row
      D hD h'.classical_bound u' v' hk' hU0' hV0' hU' hV')
  · have ha : sortedCounts (leftCount u)=[5,1,1,1] := congrArg Prod.fst h11
    have hb : sortedCounts (rightCount v)=[1,1,1] := congrArg Prod.snd h11
    have hweight : (CoverWeight.support (GeneralLP.join u v)).card=11 := by
      rw [support_weight_counts,← sortedCounts_sum (leftCount u),← sortedCounts_sum (rightCount v)
,ha,hb]
      rfl
    have hD : 10<D := by omega
    obtain ⟨j₀,hj₀,hjrest⟩ := eleven_left_marker (leftCount u) ha
    have hV1 (i : Fin 3) : rightCount v i=1 := by
      have hm := (sortedCounts_mem (rightCount v) (rightCount v i)).mpr ⟨i,rfl⟩
      rw [hb] at hm
      simpa using hm
    let r : Equiv.Perm (Fin 3) := Equiv.refl _
    let c : Equiv.Perm (Fin 4) := Equiv.swap 3 j₀
    have hc3 : c 3=j₀ := Equiv.swap_apply_left _ _
    let e' := normPermE e r c
    let u' := normPermU e r c u
    let v' := normPermV e r c v
    have h' : DistanceCriterion e' D := normPerm_criterion e D h r c
    have hk' := normPerm_kernel e r c u v hk
    have hw' : (CoverWeight.support (GeneralLP.join u' v')).card≤11 := by
      change (CoverWeight.support (GeneralLP.join (normPermU e r c u) (normPermV e r c v))).card≤11
      rw [normPerm_weight]
      omega
    have hU3' : leftCount u' 3=5 := by
      change leftCount (normPermU e r c u) 3=5
      rw [normPerm_left_count,hc3]
      exact hj₀
    have hU' (j : Fin 3) : leftCount u' (lowColumn j)=1 := by
      change leftCount (normPermU e r c u) (lowColumn j)=1
      rw [normPerm_left_count]
      apply hjrest
      intro he
      have hj : lowColumn j=3 := c.injective (he.trans hc3.symm)
      have hv := congrArg (fun z : Fin 4 => z.val) hj
      change j.val=3 at hv
      have := j.isLt
      omega
    have hV' (i : Fin 3) : rightCount v' i=1 := by
      change rightCount (normPermV e r c v) i=1
      rw [normPerm_right_count]
      exact hV1 _
    have hs := source_canonical_eleven_stabilizer e' h'.no_four h'.no_six h'.row_zero h'.column_zero
      h'.no_column_partition h'.no_triple_row D hD h'.classical_bound u' v' hk' hw' hU' hU3' hV'
    exact (normPerm_span e r c u v).mp hs
· have hv : v=0 := right_zero_of_count_sum_zero v (by omega)
  subst v
  have hu := pure_left_zero_below_classical e D h.classical_bound u hk hwD
  rw [hu]
  exact zero_full_source_span e
```

```
  · have hu : u=0 := left_zero_of_count_sum_zero u (by omega)
    subst u
    have hv := pure_right_zero_below_transpose e D h.transpose_bound v hk hwD
    rw [hv]
    exact zero_full_source_span e
```

H Pauli union weight and the literal minimum theorem

Both sectors and the entire product of stabilizer spans are retained. The public theorem discharges finite coverage before passing to literal minima.

`PauliBridge`: pauli_bound_of_x_bound

Complete checked declaration at the immutable proof commit.

```
theorem pauli_bound_of_x_bound (e : Fin 3 → Fin 4 → ZMod L) (b : ℕ)
    (hbound : ∀ x, pureX e x → b≤(CoverWeight.support x).card)
    (x z : Qubit L → ZMod 2)
    (hx : ∀ c, (∑ q,GeneralLP.HZ e c q*x q)=0)
    (hz : ∀ c, (∑ q,GeneralLP.HX e c q*z q)=0)
    (hn : ¬(fullSpanX e x ∧ fullSpanZ e z)) :
    b≤((CoverWeight.support x) ∪ (CoverWeight.support z)).card := by
  by_cases hs : fullSpanX e x
  · have hzn : ¬fullSpanZ e z := fun h => hn ⟨hs,h⟩
    exact (z_bound_of_x_bound e b hbound z ⟨hz,hzn⟩).trans
      (Finset.card_le_card Finset.subset_union_right)
  · exact (hbound x ⟨hx,hs⟩).trans (Finset.card_le_card Finset.subset_union_left)
```

`ThroughEleven`: source_pauli_bound_twelve

Complete checked declaration at the immutable proof commit.

```
theorem source_pauli_bound_twelve
    (e : Fin 3 → Fin 4 → ZMod L)
    (h4 : TannerSix.NoFour (CEdge e)) (h6 : TannerSix.NoSix (CEdge e))
    (hR0 : ∀ j,e 0 j=0) (hC0 : ∀ i,e i 0=0)
    (hCP : NoColumnPartition e) (hP : NoTripleRow e)
    (D : ℕ) (hC : ClassicalAtLeast e D) (hT : TransposeAtLeast e D)
    (x z : Qubit L → ZMod 2)
    (hx : ∀ c,(∑ q,GeneralLP.HZ e c q*x q)=0)
    (hz : ∀ c,(∑ q,GeneralLP.HX e c q*z q)=0)
    (hn : ¬(fullSpanX e x ∧ fullSpanZ e z)) :
    min D 12≤((CoverWeight.support x)∪(CoverWeight.support z)).card :=
  source_pauli_bound_twelve_of_coverage complete_coverage10 e D
    ⟨h4,h6,hR0,hC0,hCP,hP,hC,hT⟩ x z hx hz hn
```

`DistanceMinima`: source_classical_minimum_realized

Complete checked declaration at the immutable proof commit.

```
theorem source_classical_minimum_realized (e : Fin 3 → Fin 4 → ZMod L) :
    sourceClassicalHasWeight e (sourceClassicalDistance e) :=
  Nat.find_spec (source_classical_weight_exists e)
```

`DistanceMinima`: source_classical_minimum_lower

Complete checked declaration at the immutable proof commit.

```
theorem source_classical_minimum_lower (e : Fin 3 → Fin 4 → ZMod L) :
    ClassicalAtLeast e (sourceClassicalDistance e) := by
  intro x hk hn
```

```
  exact Nat.find_min' (source_classical_weight_exists e) ⟨x,hk,hn,rfl⟩
```

DistanceMinima: source_transpose_zero_kernel_infinity

Complete checked declaration at the immutable proof commit.

```
theorem source_transpose_zero_kernel_infinity (e : Fin 3 → Fin 4 → ZMod L)
    (hz : ∀ y : Fin 3 × ZMod L → ZMod 2,
      (∀ q,(∑ a,GeneralLP.C e a q*y a)=0) → y=0) : sourceTransposeDistance e=⊤ := by
  have hempty : {n | sourceTransposeHasWeight e n}=(∅ : Set ℕ∞) := by
    ext n
    constructor
    · rintro ⟨y,hk,hn,he⟩
      exact False.elim (hn (hz y hk))
    · intro hn
      exact False.elim (Set.notMem_empty n hn)
  unfold sourceTransposeDistance
  rw [hempty,sInf_empty]
```

DistanceMinima: source_quantum_minimum_lower_iff

Complete checked declaration at the immutable proof commit.

```
theorem source_quantum_minimum_lower_iff (e : Fin 3 → Fin 4 → ZMod L) (b : ℕ) :
    (b : ℕ∞)≤sourceQuantumDistance e ↔
      ∀ x z : Qubit L → ZMod 2,
        (∀ c,(∑ q,GeneralLP.HZ e c q*x q)=0) →
        (∀ c,(∑ q,GeneralLP.HX e c q*z q)=0) →
        ¬(fullSpanX e x ∧ fullSpanZ e z) →
        b≤((CoverWeight.support x)∪(CoverWeight.support z)).card := by
  constructor
  · intro hb x z hx hz hn
    have hq : sourceQuantumDistance e≤(((CoverWeight.support x)∪(CoverWeight.support z)).card : ℕ∞)
    :=
      sInf_le (show sourceQuantumHasWeight e _ from ⟨x,z,hx,hz,hn,rfl⟩)
    exact_mod_cast hb.trans hq
  · intro h
    apply le_sInf
    rintro n ⟨x,z,hx,hz,hn,rfl⟩
    exact_mod_cast h x z hx hz hn
```

CriterionScope: no_full_row_of_no_triple

Complete checked declaration at the immutable proof commit.

```
theorem no_full_row_of_no_triple (e : Fin 3 → Fin 4 → ZMod L) (h : NoTripleRow e) :
    NoFullRowRCPC e := by
  intro i j k hij hik hjk hz
  apply h i j k hij hik hjk 0 1 2 (by decide) (by decide) (by decide)
  rw [hz 0,hz 1,hz 2]
  exact ⟨rfl,rfl⟩
```

`DistanceTwelve`: source_distance_bound_twelve

Complete checked declaration at the immutable proof commit.

```
theorem source_distance_bound_twelve
    (e : Fin 3 → Fin 4 → ZMod L)
    (h4 : TannerSix.NoFour (CEdge e)) (h6 : TannerSix.NoSix (CEdge e))
    (hR0 : ∀ j,e 0 j=0) (hC0 : ∀ i,e i 0=0)
    (hCP : NoColumnPartition e) (hP : NoTripleRow e)
    (hT : (sourceClassicalDistance e : ℕ∞)≤sourceTransposeDistance e) :
    ((min (sourceClassicalDistance e) 12 : ℕ) : ℕ∞)≤sourceQuantumDistance e := by
  apply (source_quantum_minimum_lower_iff e _).mpr
  exact source_pauli_bound_twelve e h4 h6 hR0 hC0 hCP hP (sourceClassicalDistance e)
    (source_classical_minimum_lower e) (source_transpose_comparison_lower e _ hT)
```

I Complete research-source link index

Every module in the final theorem and source-scope audit is listed below. Each row has a link to the master branch and an immutable link at commit `3ae21c0719cedbd928f4f7882f9b9e266567605e`. The source archive includes these exact files and their matching build records. The accompanying HTML guide provides hashes and imports.

Module	Current	Immutable
truncated12/PackedTables.lean	source	source
truncated12/CatalogChecker.lean	source	source
truncated12/SparseCatalogChecker.lean	source	source
truncated12/FastCatalogChecker.lean	source	source
truncated12/catalog-chunks/F10_000.lean	source	source
truncated12/catalog-chunks/F10_001.lean	source	source
truncated12/catalog-chunks/F10_002.lean	source	source
truncated12/catalog-chunks/F10_003.lean	source	source
truncated12/catalog-chunks/F10_004.lean	source	source
truncated12/catalog-chunks/F10_005.lean	source	source
truncated12/catalog-chunks/F10_006.lean	source	source
truncated12/catalog-chunks/F10_007.lean	source	source
truncated12/catalog-chunks/F10_008.lean	source	source
truncated12/catalog-chunks/F10_009.lean	source	source
truncated12/catalog-chunks/F10_010.lean	source	source
truncated12/catalog-chunks/F10_011.lean	source	source
truncated12/catalog-chunks/F10_012.lean	source	source
truncated12/catalog-chunks/F10_013.lean	source	source
truncated12/catalog-chunks/F10_014.lean	source	source
truncated12/catalog-chunks/F10_015.lean	source	source
truncated12/catalog-chunks/F10_016.lean	source	source
truncated12/catalog-chunks/F10_017.lean	source	source
truncated12/catalog-chunks/F10_018.lean	source	source
truncated12/catalog-chunks/F10_019.lean	source	source
truncated12/catalog-chunks/F10_020.lean	source	source
truncated12/catalog-chunks/F10_021.lean	source	source
truncated12/catalog-chunks/F10_022.lean	source	source
truncated12/catalog-chunks/F10_023.lean	source	source
truncated12/catalog-chunks/F10_024.lean	source	source
truncated12/catalog-chunks/F10_025.lean	source	source
truncated12/catalog-chunks/F10_026.lean	source	source
truncated12/catalog-chunks/F10_027.lean	source	source
truncated12/catalog-chunks/F10_028.lean	source	source
truncated12/catalog-chunks/F10_029.lean	source	source
truncated12/catalog-chunks/F10_030.lean	source	source
truncated12/catalog-chunks/F10_031.lean	source	source
truncated12/catalog-chunks/F10_032.lean	source	source
truncated12/catalog-chunks/F10_033.lean	source	source
truncated12/catalog-chunks/F10_034.lean	source	source
truncated12/catalog-chunks/F10_035.lean	source	source
truncated12/catalog-chunks/F10_036.lean	source	source
truncated12/catalog-chunks/F10_037.lean	source	source
truncated12/catalog-chunks/F10_038.lean	source	source
truncated12/catalog-chunks/F10_039.lean	source	source
truncated12/catalog-chunks/F10_040.lean	source	source
truncated12/catalog-chunks/F10_041.lean	source	source
truncated12/catalog-chunks/F10_042.lean	source	source
truncated12/catalog-chunks/F10_043.lean	source	source
truncated12/catalog-chunks/F10_044.lean	source	source
truncated12/catalog-chunks/F10_045.lean	source	source
truncated12/catalog-chunks/F10_046.lean	source	source
truncated12/catalog-chunks/F10_047.lean	source	source

Module	Current	Immutable
truncated12/catalog-chunks/F10_048.lean	source	source
truncated12/catalog-chunks/F10_049.lean	source	source
truncated12/catalog-chunks/F10_050.lean	source	source
truncated12/catalog-chunks/F10_051.lean	source	source
truncated12/catalog-chunks/F10_052.lean	source	source
truncated12/catalog-chunks/F10_053.lean	source	source
truncated12/catalog-chunks/F10_054.lean	source	source
truncated12/catalog-chunks/F10_055.lean	source	source
truncated12/catalog-chunks/F10_056.lean	source	source
truncated12/catalog-chunks/F10_057.lean	source	source
truncated12/catalog-chunks/F10_058.lean	source	source
truncated12/catalog-chunks/F10_059.lean	source	source
truncated12/catalog-chunks/F10_060.lean	source	source
truncated12/catalog-chunks/F10_061.lean	source	source
truncated12/catalog-chunks/F10_062.lean	source	source
truncated12/catalog-chunks/F10_063.lean	source	source
truncated12/catalog-chunks/F10_064.lean	source	source
truncated12/catalog-chunks/F10_065.lean	source	source
truncated12/catalog-chunks/F10_066.lean	source	source
truncated12/catalog-chunks/F10_067.lean	source	source
truncated12/catalog-chunks/F10_068.lean	source	source
truncated12/catalog-chunks/F10_069.lean	source	source
truncated12/catalog-chunks/F10_070.lean	source	source
truncated12/catalog-chunks/F10_071.lean	source	source
truncated12/catalog-chunks/F10_072.lean	source	source
truncated12/catalog-chunks/F10_073.lean	source	source
truncated12/catalog-chunks/F10_074.lean	source	source
truncated12/catalog-chunks/F10_075.lean	source	source
truncated12/catalog-chunks/F10_076.lean	source	source
truncated12/catalog-chunks/F10_077.lean	source	source
truncated12/catalog-chunks/F10_078.lean	source	source
truncated12/catalog-chunks/F10_079.lean	source	source
truncated12/catalog-chunks/F10_080.lean	source	source
truncated12/catalog-chunks/F10_081.lean	source	source
truncated12/catalog-chunks/F10_082.lean	source	source
truncated12/catalog-chunks/F10_083.lean	source	source
truncated12/catalog-chunks/F10_084.lean	source	source
truncated12/catalog-chunks/F10_085.lean	source	source
truncated12/catalog-chunks/F10_086.lean	source	source
truncated12/catalog-chunks/F10_087.lean	source	source
truncated12/catalog-chunks/F10_088.lean	source	source
truncated12/catalog-chunks/F10_089.lean	source	source
truncated12/catalog-chunks/F10_090.lean	source	source
truncated12/catalog-chunks/F10_091.lean	source	source
truncated12/catalog-chunks/F10_092.lean	source	source
truncated12/catalog-chunks/F10_093.lean	source	source
truncated12/catalog-chunks/F10_094.lean	source	source
truncated12/catalog-chunks/F10_095.lean	source	source
truncated12/catalog-chunks/F10_096.lean	source	source
truncated12/catalog-chunks/F10_097.lean	source	source
truncated12/catalog-chunks/F10_098.lean	source	source
truncated12/catalog-chunks/F10_099.lean	source	source
truncated12/catalog-chunks/F10_100.lean	source	source
truncated12/catalog-chunks/F10_101.lean	source	source
truncated12/catalog-chunks/F10_102.lean	source	source
truncated12/catalog-chunks/F10_103.lean	source	source
truncated12/catalog-chunks/F10_104.lean	source	source
truncated12/catalog-chunks/F10_105.lean	source	source
truncated12/catalog-chunks/F10_106.lean	source	source

Module	Current	Immutable
truncated12/catalog-chunks/F10_107.lean	source	source
truncated12/catalog-chunks/F10_108.lean	source	source
truncated12/catalog-chunks/F10_109.lean	source	source
truncated12/catalog-chunks/F10_110.lean	source	source
truncated12/catalog-chunks/F10_111.lean	source	source
truncated12/catalog-chunks/F10_112.lean	source	source
truncated12/catalog-chunks/F10_113.lean	source	source
truncated12/catalog-chunks/F10_114.lean	source	source
truncated12/catalog-chunks/F10_115.lean	source	source
truncated12/catalog-chunks/F10_116.lean	source	source
truncated12/catalog-chunks/F10_117.lean	source	source
truncated12/catalog-chunks/F10_118.lean	source	source
truncated12/catalog-chunks/F10_119.lean	source	source
truncated12/catalog-chunks/F10_120.lean	source	source
truncated12/catalog-chunks/F10_121.lean	source	source
truncated12/catalog-chunks/F10_122.lean	source	source
truncated12/catalog-chunks/F10_123.lean	source	source
truncated12/catalog-chunks/F10_124.lean	source	source
truncated12/Catalog10.lean	source	source
truncated12/CoverageCore.lean	source	source
truncated12/Coverage10.lean	source	source
FamilyWords.lean	source	source
CoverWeight.lean	source	source
SourceModel.lean	source	source
GeneralLP.lean	source	source
SmallStopping.lean	source	source
CyclicCover.lean	source	source
GeneralLPPhysical.lean	source	source
TannerSix.lean	source	source
LowWeightChecks.lean	source	source
truncated12/ActualIncidence.lean	source	source
truncated12/NormalGraph.lean	source	source
truncated12/BlockParity.lean	source	source
truncated12/BlockCounts.lean	source	source
truncated12/SourceNormal.lean	source	source
truncated12/BucketCounts.lean	source	source
truncated12/TorsionClassical.lean	source	source
truncated12/IntegerSoundness.lean	source	source
truncated12/SingletonLabels.lean	source	source
truncated12/SupportSingletons.lean	source	source
truncated12/SupportPairs.lean	source	source
truncated12/PairMatching.lean	source	source
truncated12/KernelCatalog.lean	source	source
truncated12/MatchingRows.lean	source	source
truncated12/GaugeTransport.lean	source	source
truncated12/Normalization.lean	source	source
truncated12/SourceCycleFacts.lean	source	source
truncated12/PureDistance.lean	source	source
truncated12/CatalogTables.lean	source	source
truncated12/PackedTablesAudit.lean	source	source
truncated12/TargetObstructions.lean	source	source
truncated12/PatternRealizations.lean	source	source
truncated12/GeneralSymmetry.lean	source	source
truncated12/SpanCoefficients.lean	source	source
truncated12/RowCancellation.lean	source	source
truncated12/ConstantRows.lean	source	source
truncated12/FiniteLabels.lean	source	source
truncated12/catalog-chunks/F7_000.lean	source	source
truncated12/Catalog7.lean	source	source

Module	Current	Immutable
truncated12/SourceSeven.lean	source	source
truncated12/SocketEdges.lean	source	source
truncated12/SocketArithmetic.lean	source	source
truncated12/BucketSums.lean	source	source
truncated12/LocalMantel.lean	source	source
truncated12/CountPatterns.lean	source	source
truncated12/SourceProfiles.lean	source	source
truncated12/PauliBridge.lean	source	source
truncated12/ThroughNine.lean	source	source
truncated12/TenGeometry.lean	source	source
truncated12/TenIndex.lean	source	source
truncated12/TenRealization.lean	source	source
truncated12/TenEncoding.lean	source	source
truncated12/SourceTen.lean	source	source
truncated12/ExceptionalGraph.lean	source	source
truncated12/SourcePartners.lean	source	source
truncated12/K23Numbering.lean	source	source
truncated12/K23Colors.lean	source	source
truncated12/K23MissingRows.lean	source	source
truncated12/K23LeafMatches.lean	source	source
truncated12/ElevenGeometry.lean	source	source
truncated12/ElevenIndex.lean	source	source
truncated12/ElevenRealization.lean	source	source
truncated12/ElevenEncoding.lean	source	source
truncated12/catalog-chunks/F11_000.lean	source	source
truncated12/catalog-chunks/F11_001.lean	source	source
truncated12/catalog-chunks/F11_002.lean	source	source
truncated12/catalog-chunks/F11_003.lean	source	source
truncated12/catalog-chunks/F11_004.lean	source	source
truncated12/catalog-chunks/F11_005.lean	source	source
truncated12/catalog-chunks/F11_006.lean	source	source
truncated12/catalog-chunks/F11_007.lean	source	source
truncated12/catalog-chunks/F11_008.lean	source	source
truncated12/catalog-chunks/F11_009.lean	source	source
truncated12/catalog-chunks/F11_010.lean	source	source
truncated12/catalog-chunks/F11_011.lean	source	source
truncated12/catalog-chunks/F11_012.lean	source	source
truncated12/catalog-chunks/F11_013.lean	source	source
truncated12/catalog-chunks/F11_014.lean	source	source
truncated12/catalog-chunks/F11_015.lean	source	source
truncated12/catalog-chunks/F11_016.lean	source	source
truncated12/catalog-chunks/F11_017.lean	source	source
truncated12/catalog-chunks/F11_018.lean	source	source
truncated12/catalog-chunks/F11_019.lean	source	source
truncated12/catalog-chunks/F11_020.lean	source	source
truncated12/catalog-chunks/F11_021.lean	source	source
truncated12/catalog-chunks/F11_022.lean	source	source
truncated12/catalog-chunks/F11_023.lean	source	source
truncated12/catalog-chunks/F11_024.lean	source	source
truncated12/catalog-chunks/F11_025.lean	source	source
truncated12/catalog-chunks/F11_026.lean	source	source
truncated12/catalog-chunks/F11_027.lean	source	source
truncated12/catalog-chunks/F11_028.lean	source	source
truncated12/catalog-chunks/F11_029.lean	source	source
truncated12/catalog-chunks/F11_030.lean	source	source
truncated12/catalog-chunks/F11_031.lean	source	source
truncated12/catalog-chunks/F11_032.lean	source	source
truncated12/catalog-chunks/F11_033.lean	source	source
truncated12/catalog-chunks/F11_034.lean	source	source

Module	Current	Immutable
truncated12/catalog-chunks/F11_035.lean	source	source
truncated12/catalog-chunks/F11_036.lean	source	source
truncated12/catalog-chunks/F11_037.lean	source	source
truncated12/catalog-chunks/F11_038.lean	source	source
truncated12/catalog-chunks/F11_039.lean	source	source
truncated12/catalog-chunks/F11_040.lean	source	source
truncated12/catalog-chunks/F11_041.lean	source	source
truncated12/catalog-chunks/F11_042.lean	source	source
truncated12/catalog-chunks/F11_043.lean	source	source
truncated12/catalog-chunks/F11_044.lean	source	source
truncated12/catalog-chunks/F11_045.lean	source	source
truncated12/catalog-chunks/F11_046.lean	source	source
truncated12/catalog-chunks/F11_047.lean	source	source
truncated12/catalog-chunks/F11_048.lean	source	source
truncated12/catalog-chunks/F11_049.lean	source	source
truncated12/catalog-chunks/F11_050.lean	source	source
truncated12/catalog-chunks/F11_051.lean	source	source
truncated12/catalog-chunks/F11_052.lean	source	source
truncated12/catalog-chunks/F11_053.lean	source	source
truncated12/catalog-chunks/F11_054.lean	source	source
truncated12/catalog-chunks/F11_055.lean	source	source
truncated12/catalog-chunks/F11_056.lean	source	source
truncated12/catalog-chunks/F11_057.lean	source	source
truncated12/catalog-chunks/F11_058.lean	source	source
truncated12/Catalog11.lean	source	source
truncated12/Coverage11.lean	source	source
truncated12/CatalogExclusion11.lean	source	source
truncated12/SourceEleven.lean	source	source
truncated12/PermuteSource.lean	source	source
truncated12/ClassicalTransport.lean	source	source
truncated12/EntryTestTransport.lean	source	source
truncated12/GaugeWeight.lean	source	source
truncated12/GroupTransport.lean	source	source
truncated12/ProfileTransport.lean	source	source
truncated12/CountCanonical.lean	source	source
truncated12/TwelveConditional.lean	source	source
truncated12/ThroughEleven.lean	source	source
truncated12/DistanceMinima.lean	source	source
truncated12/DistanceTwelve.lean	source	source
truncated12/CriterionScope.lean	source	source